

Security Audit

Frenly 2nd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	12
Audit Resources	12
Dependencies	12
Severity Definitions	13
Status Definitions	14
Audit Findings	15
Centralisation	25
Conclusion	26
Our Methodology	27
Disclaimers	29
About Hashlock	30

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Frenly team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

FRENLY is a smart-contract-based wallet solution that helps crypto projects distribute tokens to team members, influencers, or partners in a secure and market-friendly way. It allows projects to maintain custody from day one while enforcing automatic sell limits—both per transaction and per day—to prevent sudden dumps and protect price stability. Compatible with networks like Ethereum, Polygon, and Arbitrum (with Solana coming soon), FRENLY offers flexibility through its built-in DEX and integration with PoolParty for full liquidity when needed, making it an ideal tool for responsible token distribution.

Project Name: Frenly

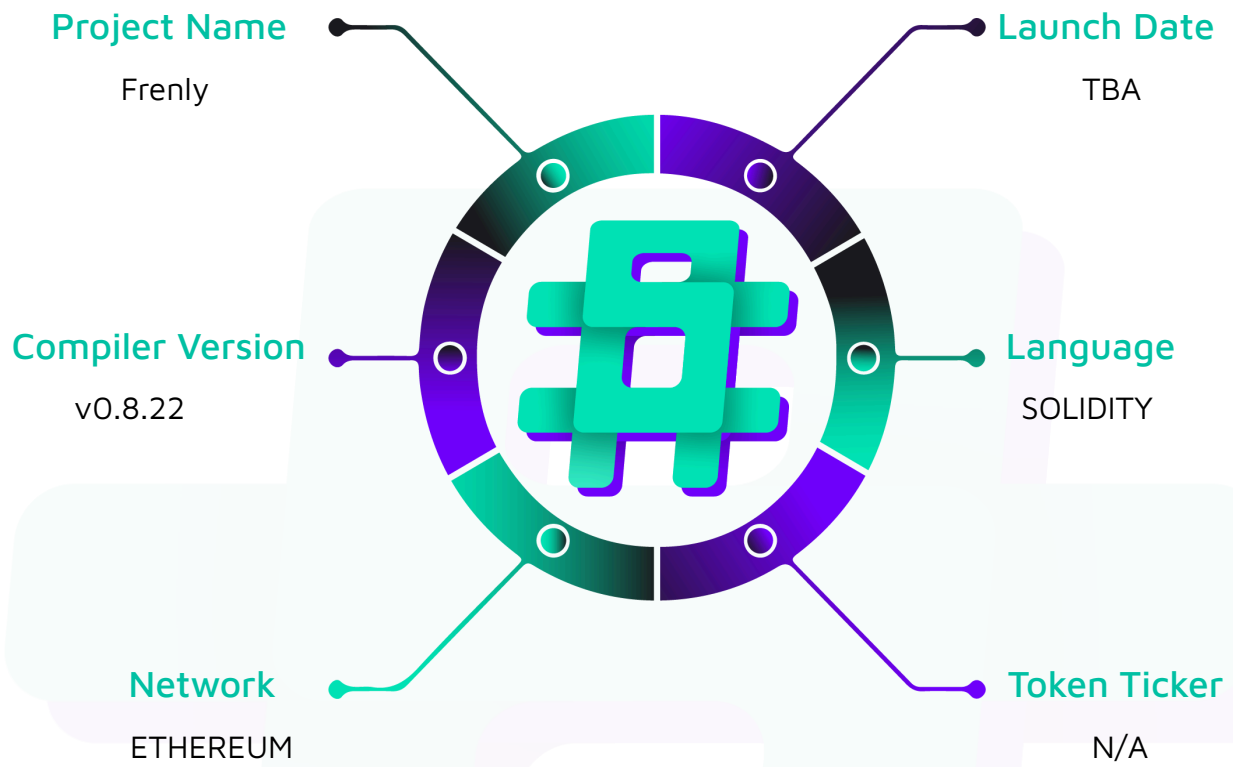
Project Type: dApp

Compiler Version: v0.8.22

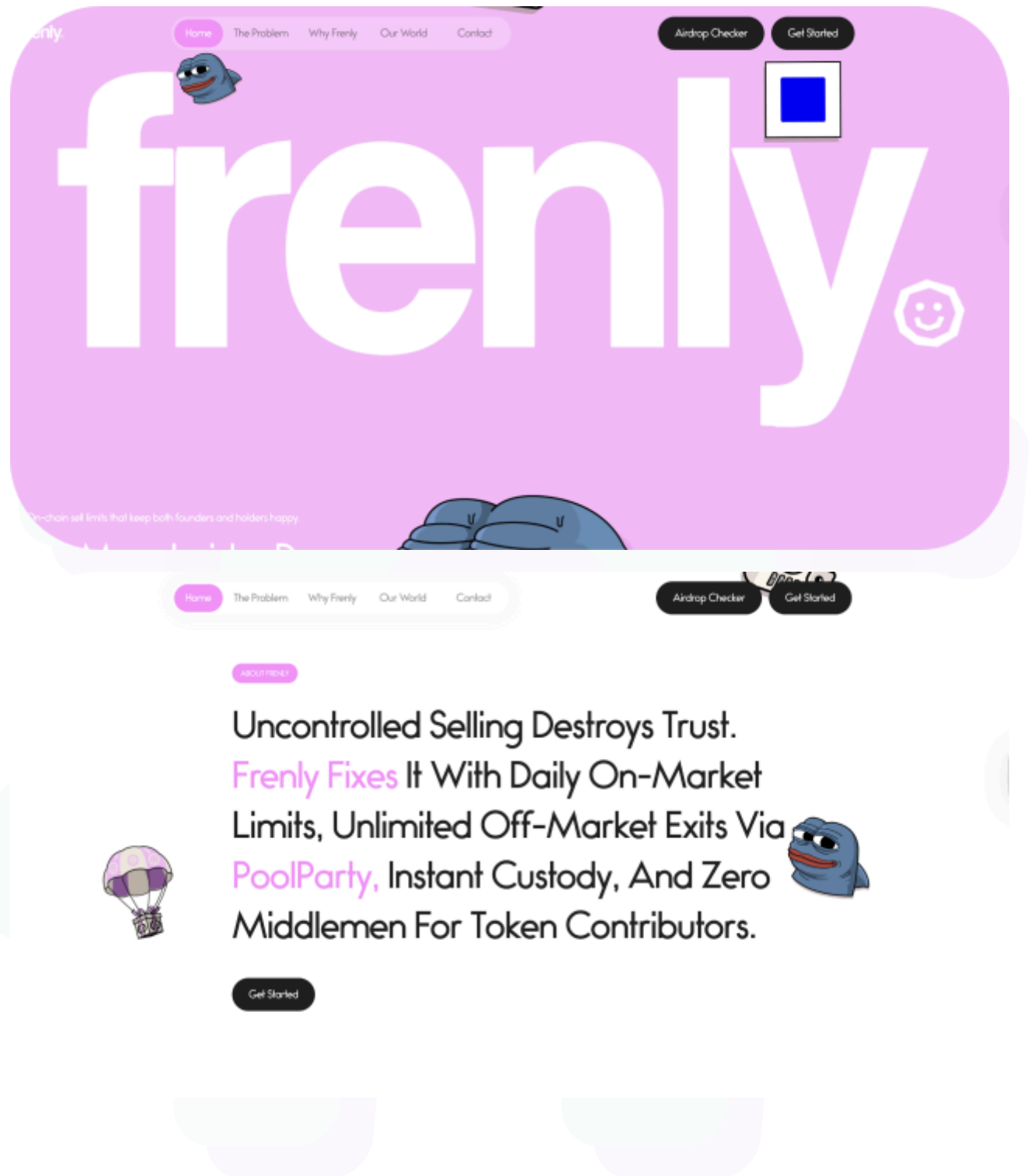
Website: <https://www.getfrenly.com/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Frenly project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Frenly Smart Contracts
Platform	Ethereum / Solidity
Audit Date	December, 2025
Contract 1	Factory.sol
Contract 2	Vault.sol
Contract 3	ChainAddressRegistry.sol
Contract 4	ENSConfig.sol
Contract 5	ENSManagementLib.sol
Contract 6	NameGenerationLib.sol
Contract 7	VaultDeploymentLib.sol
Contract 8	VaultManagementLib.sol
Contract 9	SwapValidationLib.sol
Contract 10	StakingManagementLib.sol
Audited GitHub Commit Hash	274fbc5edaef8abf3d42e414a0ac7add0e27ee14
Fix Review GitHub Commit Hash	3bfa523f0130631d4c2cc838e238e0c3a2676101

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

1 Medium severity vulnerabilities

3 Low severity vulnerabilities

3 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
FrenlyFactory.sol User Functionalities: - Create one or more vaults with parameters - Deposit tokens to an existing vault (only creator) - Execute sell from vault (only influencer) - Claim ETH from OTC pool (only influencer) - Cancel OTC pool (only influencer) - Propose vault ownership transfer (only current influencer) - Accept vault ownership transfer (only proposed influencer) - Cancel vault ownership transfer (only current influencer) - Stake PartyX tokens in a reseller pool (only influencer) - Stake unstaked SPLASH to a reseller pool (only influencer) - Move SPLASH between reseller pools (only influencer) - Initiate PartyX unstake (only influencer) - Claim unstaked PartyX (only influencer) - Unstake SPLASH from multiple reseller pools (only influencer) - Unstake all SPLASH to the vault (only influencer) - Unstake all SPLASH and initiate PartyX unstake (only influencer) - View the next vault name for a base name	Contract achieves this functionality.

- View all vaults created by a creator
- View all vaults associated with an influencer
- View the total number of vaults
- View the vault address by index
- View if ENS is enabled
- View vault ownership transfer status
- View vault staking status
- Check if the PoolParty staking pool is open
- View the reseller pools a vault has staked to

Admin Functionalities (UPGRADER_ROLE)

- Authorize contract upgrade

Owner Functionalities (DEFAULT_ADMIN_ROLE)

- Update the vault implementation address
- Update ENS node
- Set PoolParty staking contract and PartyX token addresses

FrenlyVault.sol

User Functionalities:

- View token price in ETH
- View token price in USD
- View the remaining initials available
- View initial status
- View if a proposed sale amount is allowed
- View ownership transfer status
- View SPLASH staked in a specific reseller pool
- View staking status
- View SPLASH staked in a specific reseller pool

Contract achieves this functionality.

Admin Functionalities (DEFAULT_ADMIN_ROLE) - Sell tokens or create an OTC pool - Claim ETH from the OTC pool - Cancel the OTC pool - Propose ownership transfer - Accept ownership transfer - Cancel the ownership transfer - Stake PartyX in the reseller pool - Stake unstaked SPLASH to the reseller pool - Move SPLASH between reseller pools - Initiate PartyX unstake - Claim unstaked PartyX - Unstake SPLASH from multiple pools - Unstake all SPLASH to the vault - Unstake all SPLASH and initiate PartyX unstake	
--	--

Code Quality

This audit scope involves the smart contracts of the Frenly project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Frenly project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore, it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Factory&NameGenerationLib#createVault&generateVaultName - ENS/vault name collision due to normalization mismatch

Description

The Vault names are generated from a sanitized, lowercased version of the user input, but the counter and uniqueness checks use the raw input. Different inputs that normalize to the same string can produce the same final name.

The factory uses `nameCounters[params[i].ensName]` (raw input) to advance the counter, while `NameGenerationLib` first sanitizes and lowercases the name before building the final label. Inputs like "Foo" and "foo" share the same normalized label but increment separate counters, allowing the second creation to generate the same final name (e.g., "foo-vault-1"). When writing `ensDomainToVault[vaultName]`, the mapping is overwritten, and ENS resolver updates can point an existing name to a new vault.

Impact

An attacker can front-run or follow a targeted creation to take over the same normalized name, causing users and indexers to resolve the attacker's vault instead of the original. This breaks name uniqueness guarantees and can redirect deposits or monitoring to the wrong address.

Recommendation

Normalize the name (sanitize + lowercase) once, and use that normalized string both for counter tracking and for `ensDomainToVault` keys. Before registering a subdomain or writing `ensDomainToVault`, require that the normalized vault name is unused, and revert on duplicates. Persist a `usedName[normalized]` boolean to enforce uniqueness over time.

Status

Resolved

[H-02] Vault#sell - Rollover daily limit is zero on day 1, causing false reverts

Description

With rollover enabled, the first day's effective daily limit can be zero because `lastSellDay` is set during initialization, and the per-day rollover bucket is never seeded for that day.

`initialize()` sets `lastSellDay = _currentDay()`. In `_calculateEffectiveDailyLimit(today)`, when `today == lastSellDay` it returns `dailyRolloverLimit[today]` (or `dailyRolloverLimitUsd[today]`), which are zero by default on day 1. The sell path only sees those buckets when `today != lastSellDay`, so on day 1, the enforcement compares against zero and reverts with `ExceedsDailyLimit`, unless the entire sale fits within the "initials" allowance.

Impact

Legitimate sells on the first day can revert even when within configured limits. This creates a confusing "works sometimes" behavior (dependent on initials) and can block operations right after deployment.

Recommendation

Pick one of the safe patterns:

- Do not set `lastSellDay` during `initialize`; let the first sell compute from base limits.
- Seed today's bucket during `initialize` with the base limits (fixed or USD).
- In `_calculateEffectiveDailyLimit`, if `today == lastSellDay` and the bucket is zero, fall back to the base limits.
- Or, when enforcing for `today`, compare against the computed `effectiveLimit/effectiveLimitUsd` directly instead of the unseeded storage buckets.

Status

Resolved

[H-03] Vault#sell - Market-cap toggle uses stale state, allowing one sell without limits

Description

`sell()` caches `limitsInEffect` before calling `checkMarketCap()`, which can flip the flag. The later limit checks use the cached value, not the updated one.

`sell()` sets `bool _limitsInEffect = limitsInEffect`, then calls `checkMarketCap()` which can enable limits. The enforcement block later uses `_limitsInEffect`. If market cap just rose above the threshold, the first sell after the flip can bypass limits because the code still sees `_limitsInEffect == false` for that transaction.

Impact

There is a one-transaction window where a large sell can skip daily and per-tx limits after market cap recovers, undermining the limit policy.

Recommendation

Call `checkMarketCap()` before reading `limitsInEffect`, or re-read the flag after the check, or avoid caching and reading from storage at the point of enforcement. Add a test that flips the threshold mid-flow and verifies limits are enforced immediately.

Status

Resolved

Medium

[M-01] Vault#Claim - Cancellation can fail if the influencer cannot receive ETH

Description

During cancellation, the flow claims ETH and forwards it to the influencer. If the influencer is a contract that rejects ETH, the send reverts, and cancellation aborts.

Impact

Creators may be unable to cancel a vault that has an OTC pool because the claim step fails on ETH forwarding. This blocks recovery of tokens and leaves the system stuck until the influencer address changes or the code is upgraded

Recommendation

Use a pull-payment pattern: record claimed ETH in the vault and let the influencer withdraw later, or allow cancellation to proceed even if forwarding fails, and keep the ETH as claimable. Consider allowing an alternate payout address settable by the influencer with proper checks.

Status

Resolved

Low

[L-01] SwapValidationLib - Kyber router selector constant may be incorrect

Description

`KYBER_SWAP_SELECTOR` is set to `0xd9c45357` with a comment for `swap(SwapExecutionParams)`. This selector does not match the real router entry point.

Impact

If the selector is wrong, valid payloads can revert during validation or decode into the wrong shape, breaking sells. It likely causes denial-of-service rather than fund loss, but it will disrupt production flows.

Recommendation

Regenerate selectors from the real router ABI during build (or import a typed interface and take selectors from it). Add tests using real, known-good Kyber payloads so `validateSwapGeneric()` accepts them and rejects malformed ones.

Status

Resolved

[L-02] ChainAddressRegistry#getPriceOracleAddress - Outdated Oracle address & Oracle Addresses Mapping InConsistency

Description

Protocol is using the old ORACLE address for ZKSYNC_CHAIN_ID, which can lead to issues like stale prices, unexpected reverts, or wrong values. Additionally, the ORACLE address for all other chains except SONIC_CHAIN_ID returns the old ORACLE as well.

Recommendation

Use the same ORACLE address of SONIC_CHAIN_ID for all other chains, as the ORACLE is present on all other chains at the same address. Additionally, replace the ZKSYNC_CHAIN_ID oracle with 0x739B4e7a3ad8210B6315F75b24cfe0D3226f6945

Status

Resolved

[L-03] Vault#createOTCPool - Multiple OTC pools can be created → older pools may become unreachable

Description

Creating a new OTC pool overwrites otcPoolAddress without guarding against an existing pool, and claim/cancel only reference the single stored address.

_createOtcPool() assigns otcPoolAddress = createPool(...). If a second pool is created, the previous address is lost in the contract state. Since claim() and cancelOtcPool() only use otcPoolAddress, older pools cannot be managed through the contract anymore.

Impact

Tokens or ETH associated with a previous pool can get stuck because the vault no longer tracks that pool. Operators cannot cancel or claim from earlier pools via the standard functions.

Recommendation

Enforce a single active pool by reverting if `otcPoolAddress != address(0)`, or track pools in an `array/map` and expose `claim/cancel` by pool `id/address`. Emit events on every `create/close` and add checks that prevent accidental overwrites.

Status

Acknowledged

QA

[Q-01] Factory#depositToVault - Function utility adds little security value

Description

Anyone can send ERC20 tokens directly to the vault via `transfer/transferFrom`. The factory's `depositToVault()` primarily adds an event and a creator-only restriction.

Extra surface area without stronger guarantees; the function is optional from a security perspective.

Recommendation

Keep it only if you want the event trail and creator-only flow. If kept, state the purpose in NatSpec and ensure the event is used by indexers/ops.

Status

Acknowledged

[Q-02] Factory#updateENSConfig - Missing Validation in UpdateENSConfig

Description

`updateENSConfig()` sets registry and resolver without checking for zero address or code presence.

An admin mistake (or a compromised admin key) can set these to EOAs or zero, bricking ENS features and reverse records until fixed, and possibly confusing indexers and users.

Recommendation

Require non-zero and `isContract()` for both addresses. Emit an event with `old/new` values. Consider a two-step change (`propose/accept`) for sensitive infra addresses.

Status

Resolved

[Q-03] Vault#sell - Allowance not reset to zero after swap**Description**

`sell()` force-approves the exact amount to the DEX before the swap, but does not reset to zero after the swap completes.

If the token is non-standard and does not decrease allowance as expected, or if the router spends less than approved, leftover allowance may remain. If the router is compromised later, that leftover can be abused.

Recommendation

After `_executeSwap` returns, `forceApprove(dexAddress, 0)`. Alternatively, use a permit or an execution flow that transfers tokens into the router in the same call rather than relying on approvals. Add a test that asserts allowance is zero after a successful swap.

Status

Resolved

Centralisation

The Frenly project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Frenly project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

